

Western Norway
University of
Applied Sciences

Deterministic Rounding of LP

Chapter 4

Sven-Olai Høyland

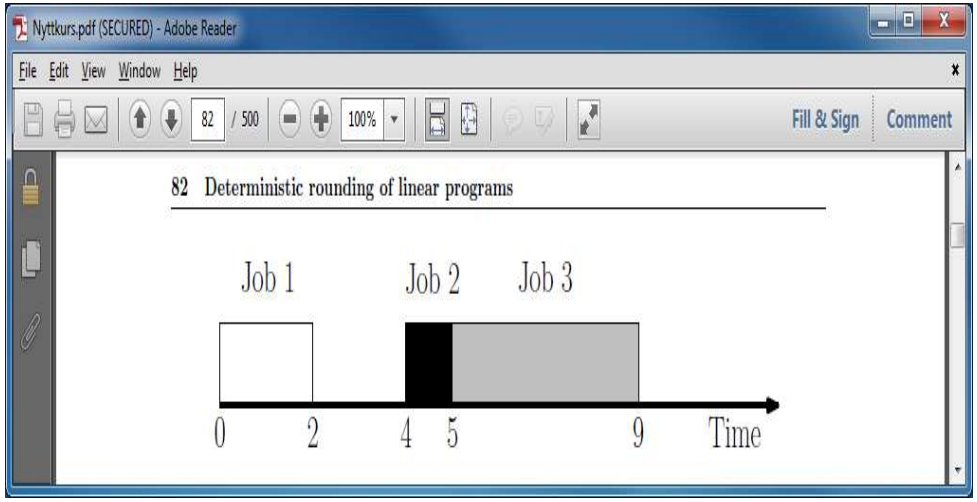
Sceduling problem

- Minimizing the Sum of Completion Times on a single machine
  - n jobs, each with a release date $r_j \geq 0$ and a processing time $p_j > 0$ (both integers)
    - Process the jobs on one machine
    - No job is processed before its release date
    - At most one job is processed at a point of time
    - The jobs are processed nonpreemptly
  - Let C_j denote the finishing time for job j.
- Goal: Minimize $\sum_{j=1}^{n} C_j$

Example

Jobb number	Release time	Processing time
1	0	2
2	4	1
3	1	4

One feasible solution



$$C_1 + C_2 + C_3 = 2 + 5 + 9 = 16$$

Possible to do better?

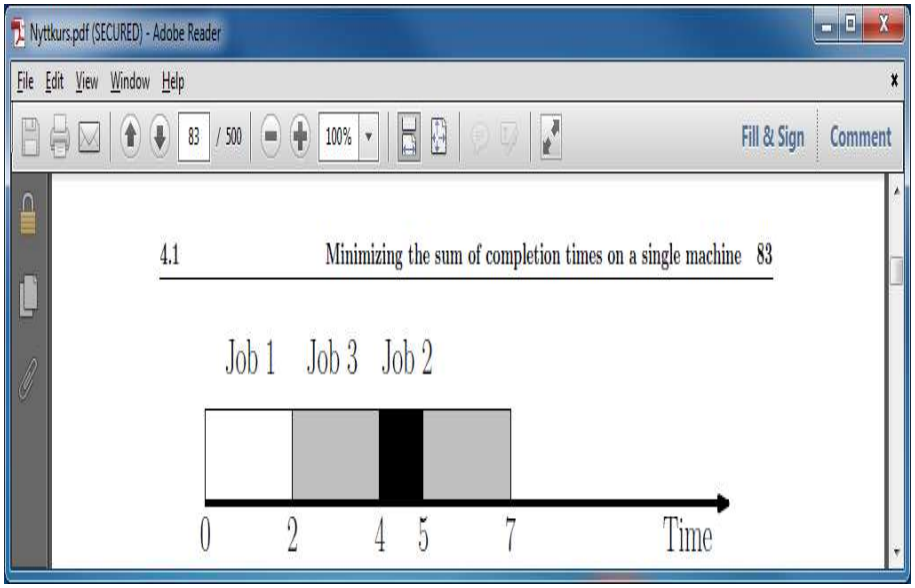
Preemptive version

- Do not need to complete a job before we start another job
- Can find an optimal solution to this problem via the shortest remaining processing time (SRPT) rule
  - while there are jobs left do
    - If there are no released jobs, move time until next job is released
    - Among released jobs that are not done, choose the one with shortest remaining processing time
    - Schedule it until it is completed or a new job is released

Preemptive example

Jobb number	Release time	Processing time
1	0	2
2	4	1
3	1	4

One feasible solution



$$C_1 + C_2 + C_3 = 2 + 5 + 7 = 14$$

Observation

- Let C_j^P be the completion time of job j in an optimal preemptive schedule and let OPT be the sum of completion times in an optimal nonpreemptive schedule.
- Observation 4.1. $\sum_{j=1}^{n} C_j^P \leq OPT$
- Proof. Follows from the fact that an optimal nonpreemptive schedule is feasible for the preemptive scheduling problem.

Algorithm

- Find an optimal preemptive schedule using SRPT
- Schedule the jobs nonpreemptly in the same order that they complete in the preemptive schedule

Proof

- Lemma 4.2. For each job $j = 1, \dots, n$, $C_j^N \leq 2C_j^P$
- Index the jobs such that $C_1^P \leq C_2^P \leq \cdots C_n^P$
- Proof. Have,
  - $C_j^P \geq \max_{k=1,\dots j} r_k$
  - $C_j^P \geq \sum_{k=1}^{j} p_k$
  - $C_j^N \geq \max_{k=1,\dots j} r_k$

Proof continued

- Consider nonpreemptive schedule
  - Any period that the machine is idle
  - Next job to be processed is not released
  - In interval from $\max_{k=1,\dots j} r_k$ to C_j^N the machine can not be idle
  - Interval ca be of length at most $\sum_{k=1}^{j} p_k$
- $C_j^N \leq \max_{k=1,\dots j} r_k + \sum_{k=1}^{j} p_k \leq 2C_j^P$

Theorem

- Theorem 4.3. Scheduling in order of the completion of an optimal preemptive schedule i a 2-approximation algorithm for scheduling a single machine with release dates to minimize the sum of completion times.
- Proof. Summing completion time for all jobs combined with Lemma 4.2. $\sum_{j=1}^{n} C_j^N \leq 2 \sum_{j=1}^{n} C_j^P \leq 2OPT$

Lemma 4.2

Chapter 4.2

- Not in curriculum

Chapter 4.3

- Solving large linear programs in polynomial time via the Ellipsoid method
  - Need a separation oracle that takes as input a supposedly feasible solution x. It either verifies that x is feasible or returns a constraint that is violated
  - The algorithm runs in polynomial-time time even if there are an exponentially many constraints

- Sufficient to know this (no details of the method).

Chapter 4.4 and 4.5

- Not in curriculum

4.6 The Bin-Packing Problem

- Given n pieces (or items) with specified sizes $a_1, a_2, \ldots, a_n$ such that $1 \geq a_1 \geq a_2, \geq \cdots \geq a_n > 0$
- Wish to pack the items into minimum number of bins such that the sum of sizes in each bin is at most 1.
- Saw in Chapter 3
  - Poly time algorithm that finds a solution with at most $(1 + \varepsilon)OPT(I) + 1$ bins
  - Will improve this

Ideas

- Integer programming
  - Approximately solved by rounding its LP relaxation
- Improved grouping scheme
  - Harmonic grouping
- Recursive application of the two

Configuration

- Group pieces of identical size
  b_1 pieces of size s_1, b_2 pieces of size s_2,…, b_m pieces of size s_m
- The contents of each bin can be described by an m-tuple $(t_1, \dots, t_m)$ where t_i indicates the number of pieces of size s_i.
- The m-tuple is a configuration if $\sum_{i=1}^{m} t_i s_i \leq 1$
- Let $T_1, \dots, T_N$ be a complete enumeration of them
- Problem: May be an exponential number of configurations.

Integer Program

- Have variable x_j that specifies number of bins of configuration T_j and $t_{ij}x_j$ pieces of size s_i

- minimize $\sum_{j=1}^{N} x_j$
- subject to
  - $\sum_{j=1}^{N} t_{ij}x_j \geq b_i, \quad i = 1, \ldots, m,$ (must pack all items)
  - $x_j \in \mathbb{N}, \qquad \mathrm{j} = 1, \ldots \mathrm{N}$

Example

- 10 pieces of size .60
- 5 pieces of size .51
- 7 pieces of size .43
- 20 pices of size .33

Configurations

0,60	0,51	0,43	0,33	Sum
1	0	0	1	0,93
1	0	0	0	0,60
0	1	1	0	0,94
0	1	0	1	0,84
0	1	0	0	0,51
0	0	2	0	0,86
0	0	1	1	0,76
0	0	1	0	0,43
0	0	0	3	0,99
0	0	0	2	0,66
0	0	0	1	0,33

Problem

- Solve the LP relaxation of the integer program with the Ellipsoid method
  - Problem: Exponential number of variables



Solve the dual problem (have exponential number of constraint). Can be done by Ellipsoid method in polynomial time

Assumption

- Can assume that the smallest piece size $s_m \geq 1/SIZE(I)$

Harmonic grouping

- Process the pieces in order of decreasing size
- Close the current group whenever its total size is at least 2.
- Number the groups $G_1, \ldots, G_r$ and let n_i be the number of items in group G_i
- Will have $n_i \geq n_{i-1}, \quad i = 2, \ldots, r-1$

Construct new instance I’

- For each $i = 2, \dots, r-1$, we put n_{i-1} pieces in I' of size equal the largest item in G_i and discards the rest
- This means we discard
  - G_1
  - G_r
  - $n_i - n_{i-1}$ items from the other groups G_i (if the number of items is equal, we discard no items)

Example (filled in lecture)

- I = {0.6, 0.59, 0.57, 0.53, 0.5, 0.49, 0.47, 0.46 0.45, 0.44, 0.43…}

  G_1 =
  G_2 =

- Start of I'

I' = {

Example (filled in)

- I = {0.6, 0.59, 0.57, 0.53, 0.5, 0.49, 0.47, 0.46, 0.45, 0.44, 0.43, …}

  G_1 = {0.6, 0.59, 0.57, 0.53}
  G_2 = {0.5, 0.49, 0.47, 0.46, 0.45}

- Start of I' (Notice that the first four items in G_2 are rounded to 0.5 and the item of size 0.45 is discarded. The in the next group the itemes are rounded to 0.44)

I' = {0.5, 0.5, 0.5, 0.5, 0.44, 0.44, …}

Lemma

- Lemma 4.15. Let I’ be the bin-packing input obtained from an input I by applying the harmonic grouping scheme. The number of distinct piece sizes in I’ is at most $\frac{SIZE(I)}{2}$; the total size of all discarded pieces is $O(\log SIZE(I))$.

Algorithm

Binpack(I)

if $SIZE(I) < 10$ then

- Pack remaining pieces using First Fit

else

- Apply harmonic grouping scheme to create instance I'
- Pack discards in $O(\log SIZE(I))$ bins using First Fit
- Let x be optimal solution to configuration LP for I'
- Pack $\lfloor x_j \rfloor$ bins in configuration T_j for $j = 1, \ldots, N$; call the packed pieces instance I_1
- Let I_2 be the remaining pieces from I' (resulting from rounding down the $x_j, j = 1, \ldots, N$)
- Binpack(I_2)

Lemma 4.16

- Lemma 4.16. For any bin-packing input I, from which the harmonic grouping scheme produces input I', which is then partitioned into I_1 and I_2, based on the integer and fractional parts of an optimal solution to the configuration linear program, we have that

$$OPT_{LP}(I_1) + OPT_{LP}(I_2) \leq OPT_{LP}(I') \leq OPT_{LP}(I)$$

Lemma 4.17

- Theorem 4.17. The recursive application of the harmonic grouping scheme yields a polynomial-time approximation algorithm for the bin-packing problem that uses $OPT_{BP}(I) + O(\log^2(OPT_{BP}(I)))$ bins.